Chapter 9

Java Beans

CHAPTER OUTLINES

To discuss basic concepts of applets:

- To understand jar files and how to create it
- To demonstrate how to read and update jar files
- To explain importance of java beans
- To differentiate java beans from java classes
- To explain BDK and its use
- To demonstrate bean writing process

9.1 JAR Files

JAR stands for Java Archive. In software, JAR file is a package file that is typically used to aggregate many Java class files and associated metadata and resources (text, images, etc.) into one file to distribute application software or libraries on the Java platform. JAR files are fundamentally archive files, built on the ZIP file format and have the .jar file extension. Computer users can create or extract JAR files using the jar command that comes with a JDK. They can also use zip tools to do so.

9.1.1 Creating a JAR File

The basic format of the command for creating a JAR file is: *jar cf jar-file input-file(s)*. The options and arguments used in this command are:

→ The *c* option indicates that we want to *create* a JAR file.

→ The *f* option indicates that we want the output to go to a *file* rather than to standard output device.

→ *jar-file* is the name that we want the resulting JAR file to have. We can use any filename for a JAR file. By convention, JAR filenames are given a .jar extension, though this is not required.

→ The *input-file(s)* argument is a space-separated list of one or more files that we want to include in our JAR file. The *input-file(s)* argument can contain the wildcard * symbol. If any of the "*input-files*" are directories, the contents of those directories are added to the JAR archive recursively.

The *c* and *f* options can appear in either order, but there must not be any space between them. This command will generate a compressed JAR file and place it in the current directory. The command will also generate a default manifest file for the JAR archive.

We can add also additional options to the *cf* options of the basic command. Including option v produces verbose output on stdout while the JAR file is being built. The verbose output tells us the name of each file as it is added to the JAR file. Option *M* indicates that the default manifest file should not be produced. Option *m* can be used when we want to include manifest information from an existing manifest file.

Examples

jar cf abc.jar xyz.class

This command creates the jar file *abc.jar* from the class file *xyz.class* in current directory.

jar cf abc.jar xyz.class images

This command creates the jar file *abc.jar* from the class file *xyz.class* in current directory. It also includes information in images folder to the jar file

9.1.2 Reading from JAR File

The basic command to use for extracting the contents of a JAR file is: *jar xf jar-file [archived-file(s)]*. Let's look at the options and arguments in this command:

→ The x option indicates that we want to *extract* files from the JAR archive.

→ The f option indicates that the JAR *file* from which files are to be extracted is specified on the command line, rather than through stdin.

→ The jar-file argument is the filename (or path and filename) of the JAR file from which to extract files.

→ *archived-file(s)* is an optional argument consisting of a space-separated list of the files to be extracted from the archive. If this argument is not present, the Jar tool will extract all the files in the archive.

As usual, the order in which the x and f options appear in the command doesn't matter, but there must not be a space between them. When extracting files, the Jar tool makes copies of the desired files and writes them to the current directory, reproducing the directory structure that the files have in the archive. The original JAR file remains unchanged.

Example

Jar xf xyz.jar

This command will extract all files and folders in xyz.jar file in current directory

Jar xf test.jar Test.class Login.class images/x.gif

This command will extract *Test.lass*, *Login.class* and *x.gif* file included in images folder in *xyz.jar* file in current directory.

9.1.3 Updating a JAR File

The Jar tool provides an option which we can use to update the contents of an existing JAR file by modifying its manifest or by adding files. The basic command for adding files has this format: *jar uf jar-file input-file(s)*. In this command:

→ The *u* option indicates that we want to *update* an existing JAR file.

→ The *f* option indicates that the JAR file to update is specified on the command line.

→ *jar-file* is the existing JAR file that is to be updated.

→ *input-file(s)* is a space-delimited list of one or more files that we want to add to the JAR file.

Any files already in the archive having the same pathname as a file being added will be overwritten.

Examples

Jar uf xyz.jar A.class

This command will update existing *xyz.jar* file and will add one more class file *A.class* in the JAR file.

Jar uf xyz.jar A.class images/ab.gif

This command will update existing *xyz.jar* file and will add one more class file *A.class* and an image named *ab.gif* residing in images folder in the JAR file.

9.2 What is a JavaBean?

A Java Bean is a reusable software component written in java that can be manipulated visually in an application builder tool. The idea is that one can start with a collection of such components, and quickly wire them together to form complex programs without actually writing any new code.

Software components must, in general, adopt standard techniques for interacting with the rest of the world. For example, all GUI components inherit the *java.awt.Component* class, which means that one can rely on them to have certain standard methods like *paint()*, *setSize()*, etc. Java Beans are not actually required to inherit a particular base class or implement a particular interface. However, they do provide support for some or all of the following key features

- Support for *introspection*. Introspection is the process by which an application builder discovers the properties, methods and events that are associated with a Java Bean.
- Support for *properties*. These are basically member variables that control the appearance or behavior of the Java Bean.
- Support for *customization* of the appearance and behavior of a Java Bean.
- Support for *events*. This is a mechanism by which Java Beans can communicate with one another.
- Support for *persistent storage*. Persistence refers to the ability to save the current state of an object, so that it can be restored at a later time.

The JavaBeans API describes the JavaBeans component architecture. Through the design mode of a builder tool we can use the Properties window of the bean to customize the bean and then save (persist) our beans using visual manipulation. We can select a bean from the toolbox, drop it into a form, modify its appearance and behavior, define its interaction with other beans, and combine it and other beans into an applet, application, or a new bean. Thus, we can say that JavaBean is a reusable & customizable Java class that must follow certain rules or conventions. These rules or conventions are called design patterns. Some of the rules are: *It must implement serializable interface, it must have one no-argument public constructor, it must have getters and setters for each attribute etc.*

9.2.1 Basic Bean Concepts

JavaBeans, regardless of their functionality, are defined by the following features.

- **Introspection**—Beans support introspection, which is the process by which a builder tool analyzes how a Bean works. They adhere to specific rules called design patterns for naming Bean features. Each Bean has a related Bean information class, which provides property, method, and event information about the Bean itself. Each Bean information class implements a *BeanInfo* interface, which explicitly lists the Bean features that are to be exposed to application builder tools.
- **Properties**—Properties control a Bean's appearance and behavior. Builder tools introspect on a Bean to discover its properties and to expose them for manipulation. As a result, we can change a Bean's property at design time.
- **Customization**—The exposed properties of a Bean can be customized at design time. Customization allows a user to alter the appearance and behavior of a Bean. Beans support customization by using property editors or by using special, sophisticated Bean customizers.
- **Events**—Beans use events to communicate with other Beans. Beans may fire events. When a Bean fires an event it is considered a source Bean. A Bean that receives an event is considered a listener Bean. A listener Bean registers its interest in the event with the source Bean. Builder tools use introspection to determine those events that a Bean sends and those events that it receives.
- **Persistence**—Beans use Java object serialization, implementing the *java.io.Serializable* interface, to save and restore state that may have changed as a result of customization. State is saved, thus when we customize a Bean in an application builder, changed properties can be restored at a later time.
- **Methods**—All JavaBean methods are identical to methods of other Java classes. Bean methods can be called by other Beans or via scripting languages. By default all public methods are exported.

While Beans are intended to be used primarily with builder tools, they need not be. Beans can be manually manipulated by text tools through programmatic interfaces.

9.2.2 Advantages of Java Beans

The following list enumerates some of the specific benefits that Java technology provided for a component developer:

- A Bean obtains all the benefits of Java's "write-once, run-anywhere" paradigm.
- The properties, events, and methods of a Bean that are exposed to an application builder tool can be controlled.
- A Bean may be designed to operate correctly in different environments, which makes it useful in global markets.

- The configuration settings of a Bean can be saved in persistent storage and restored at a later time.
- A Bean may register to receive events from other objects and can generate events that are sent to other objects.

9.2.3 What Distinguishes JavaBeans from Java Classes?

As already mentioned, A JavaBean is a software component that is written in Java programming language. It is similar to a java class except following differences:

- A JavaBean must adhere to certain conventions (mentioned above) regarding property and event interface definitions whereas java classes need not to be.
- A builder tool must be able analyze how a Bean works through introspection process. This is not necessary Java classes.
- Java Beans must be customizable. It should allow its state to be manipulated at design time. This is not necessarily to be true in case of java classes.
- JavaBeans must implement serailizable interface which is not necessary for java classes.
- Every JavaBean must have one public no-argument constructor which is not true for Java classes

In conclusion it can be said that a Bean is a Java class, but a Java class may not be a bean. Thus, bean in a specific Java class and has rules that have to be followed, called design patterns.

9.3 Beans Development Kit (BDK) and Bean Box

BDK is an application that facilates the development of JavaBeans. It is provided as free by sun. The BeanBox is a part of Beans Development Kit (BDK). Besides this BDK includes the documents, sample codes, and JAR files. Thus, BDK makes bean development much easier.

The BeanBox is an application for testing Beans, and serves as a simple demonstration for how more sophisticated Beans-aware builder tools should behave. We use the BeanBox to understand Bean characteristics. It is not designed for building GUI applications. We can create a JavaBean and then use the BeanBox to test that it runs properly. If a JavaBean runs properly in the BeanBox, you can be sure that it works properly with other commercial builder tools.

Starting the BeanBox

When we start the BeanBox, we will see three windows: *ToolBox window, BeanBox window, and Properties window*, as shown in figure below:

- The T
BeanB
starts,
the be
Toolbo
- The B
empty
other b
- The P
can us

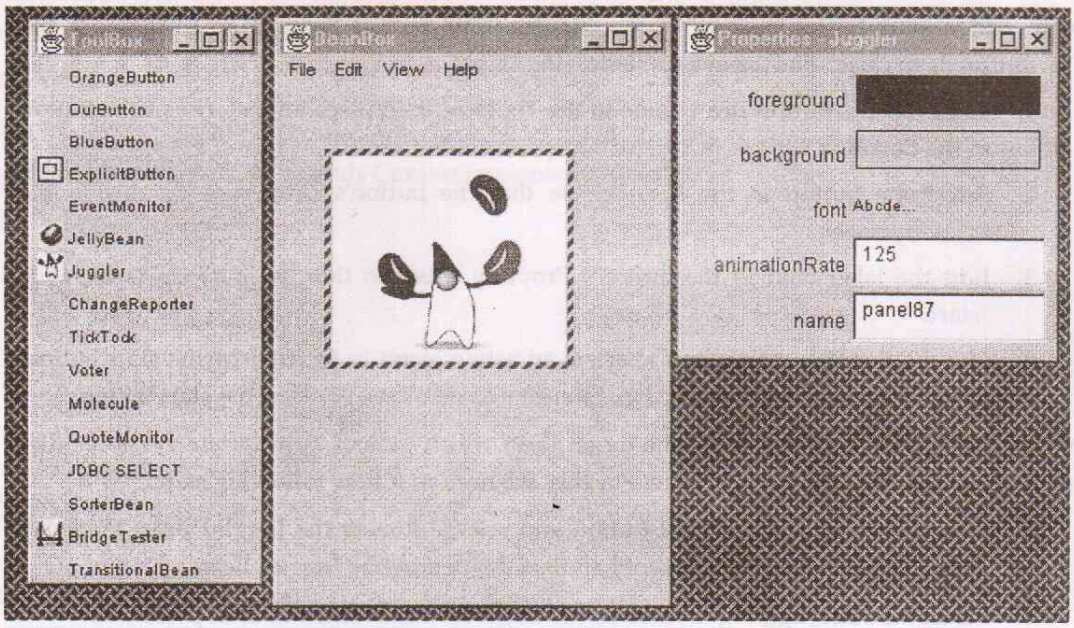
9.3.1 Using the

The BeanBox
code. As a firs
will start or sto

Creating Bean

We will start b
can be done as

1. Click
2. Place
Juggle



- **The Toolbox Window** displays the JavaBeans that are currently installed in the BeanBox, such as the Beans the come with the BeanBox demo. When the BeanBox starts, it automatically loads its Toolbox with the Beans in the JAR files contained in the bean/jars directory. We can add additional Beans, such as our own Beans, to the Toolbox.
- **The BeanBox window** itself appears initially as an empty window. We use this empty window for building applications. It is sometimes referred to as a "form" by other builder tools.
- **The Properties window** displays the current properties for the selected Bean. We can use the Properties window or sheet to edit a Bean's properties.

9.3.1 Using the BDK BeanBox

The BeanBox enables us to construct simple Beans applications without writing any Java code. As a first example, we can build a simple "Juggling Duke" application in which Duke will start or stop juggling depending on which of two buttons we push.

Creating Bean

We will start by dropping the Juggler Bean from the Toolbox into an empty BeanBox. This can be done as below:

1. Click on Juggler Bean to select from the list of Beans in the Toolbox window.
2. Place the cursor anywhere in the BeanBox, then click the mouse. This inserts a Juggler Bean into the BeanBox window.

Now add a start button to control the Juggler. This button Bean is an instance of the OurButton Bean class. For this we use following steps:

1. Click the OurButton Bean name in the Toolbox, then place an instance of the button in the BeanBox.
2. Select the button in the BeanBox so that the button's properties display in the Property sheet.
3. Edit the label field in the button's Property sheet so that the button's label reads "start."
4. Use the BeanBox edit menu to select an action event to be fired by the start button. Before choosing the event action, be sure that you have selected the start button.

Next mouse press should be over the target Bean which defines appropriate event-handler methods, in this case the Juggler Bean. For this, we need to follow following steps:

1. Drag the line from the start button and rèlease it over the Juggler Bean. A dialog appears listing applicable event handlers defined by the Juggler Bean.
2. Select the start Juggling method as the target for the event, and then press OK.

Now, when we press the start button Duke should start tossing beans around in a circle over his head like a professional juggler.

Completing the Program

To complete the example program, add a stop button. Repeat the steps that are performed for adding the start button and connect it to the appropriate Juggler action. Now, we should be able to start and stop the juggler by pressing the appropriate button.

9.4 Bean Writing Process

Writing a bean is not technically difficult—there are only a few new classes and interfaces for us. In particular, the simplest kind of a bean is nothing more than a Java class that follows some fairly strict naming conventions for its methods. Steps for writing a bean are:

1. create a directory for the new Bean
2. create the java source file(s)
3. compile the source file(s)
4. Create manifest file.
5. Generate a JAR file.
6. Start the BDK.
7. Test the bean

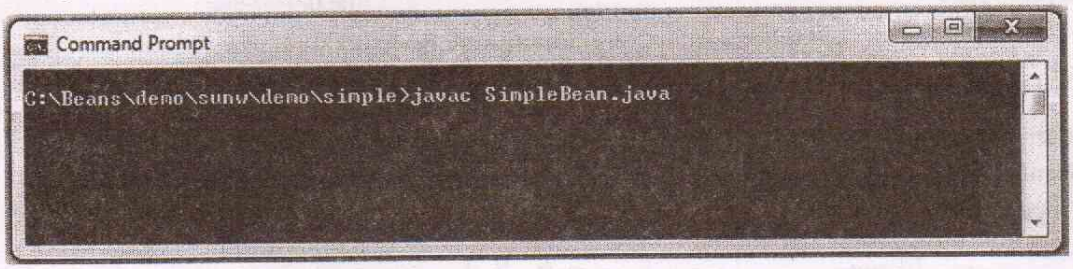
Create a directory for the new Bean

In our example C:\Beans\demo\sunw\demo\simple

Create the java source file(s)

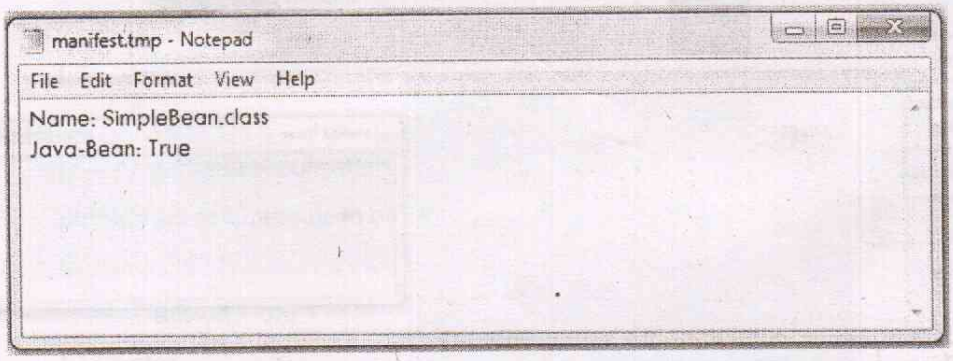
```
import java.awt.*;
import java.io.Serializable;
public class SimpleBean extends Canvas implements Serializable
{
    public SimpleBean()
    {
        setSize(60,40);
        setBackground(Color.red);
    }
}
```

Compile the source file(s)



Create manifest file

We must create the manifest file that provides information about which components in the JAR file are java beans. The manifest file must have name *manifest.tmp* or *menifest.mft*. We can put any number of files in the manifest file entry but if we have class files acting as java beans then each entry of the file must be immediately followed by the line **Java-Bean: True.** For example,



The above file indicates there is only one class file in JAR file. Be careful that we must keep new line character (i.e. press enter key) after second entry.

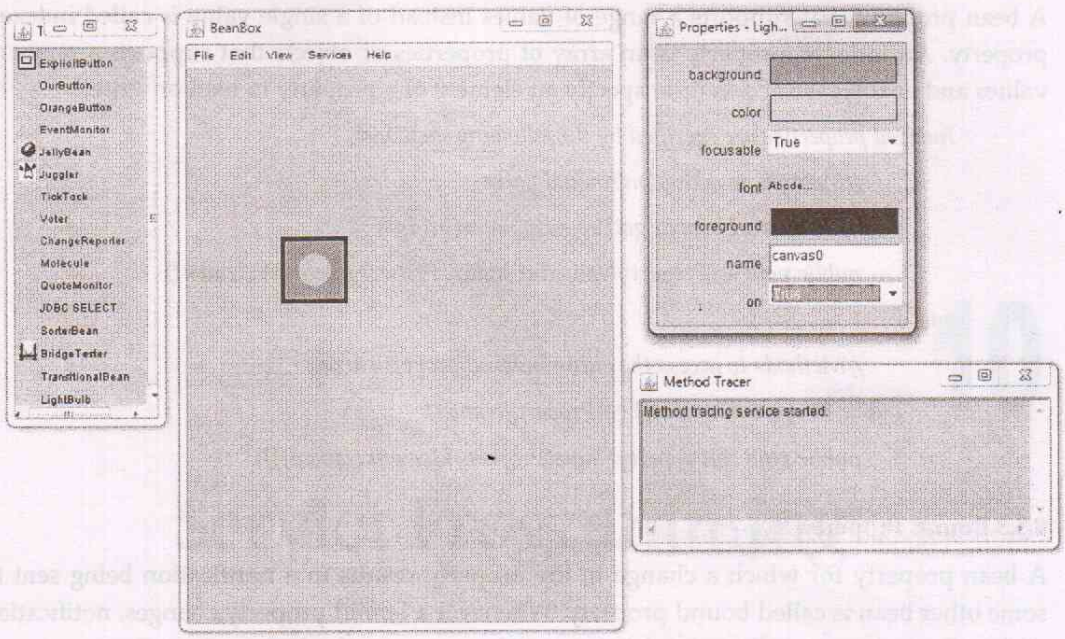
```
    // Override the paint() method to draw the LightBulb
    public void paint(Graphics g)
    {
            if (on) g.setColor(color);
            else g.setColor(COLOR_OFF);
            g.fillOval(10, 10, 30, 30);
    }

    public void switchOn()
    { // switch on the Light
            on = true;
            repaint();
    }

    public void switchOff()
    { // switch off the Light
            on = false;
            repaint();
    }

    public void toggle()
    { // If on turns off; else turns on
            on = !on;
            repaint();
    }
}
```

When we Test this bean in BDK, it looks like below:



9.5 Bean Properties

A bean property is a named attribute of a bean that can affect its behavior or appearance. Examples of bean properties include color, label, font, font size, and display size. The JavaBeans specification defines the following types of bean properties:

- *Simple Property*– A bean property with a single value whose changes are independent of changes in any other property.
- *Indexed Property* – A bean property that supports a range of values instead of a single value.
- *Bound Property*– A bean property for which a change to the property results in a notification being sent to some other bean.
- *Constrained Property*– A bean property for which a change to the property results in validation by another bean. The other bean may reject the change if it is not appropriate.

9.5.1 Simple Property

A bean property with a single value whose changes are independent of changes in any other property is called simple property. To add simple properties to a bean, add appropriate getxxx and setxxx methods (or isxxx and setxxx methods for a Boolean property). The names of these methods follow specific rules called *design patterns*.

9.5.2 Indexed Property

A bean property that supports a range of values instead of a single value is called indexed property. An indexed property is an array of properties or objects that supports a range of values and enables the access or to specify an element of a property to read or write.

Indexed properties are specified by the following methods:

```
//Methods to access individual values
public PropertyElement getPropertyName(int index)
public void setPropertyName(int index, PropertyElement element)
```

and

```
//Methods to access the entire indexed property array
public PropertyElement[] getPropertyName()
public void setPropertyName(PropertyElement element[])
```

9.5.3 Bound Property

A bean property for which a change to the property results in a notification being sent to some other bean is called bound property. Whenever a bound property changes, notification of the change is sent to interested listeners. The access methods for a bound property are defined in the same way as those for simple properties. However, you also need to provide the event listener registration methods for *PropertyChangeListener* classes and fire a *PropertyChange* event to the *PropertyChangeListener* objects by calling their *propertyChange* methods.

Exercise

1. What is meant by jar file? Explain the process of creating jar file with example.
2. Explain the process of reading and updating jar file with suitable example of each.
3. What is meant by Java bean? Explain key features of Java bean with brief description.
4. What are the advantages of using Java beans? How it is different from Java classes? Explain.
5. What is meant by BDK and Bean Box? Explain the process of using Bean Box.
6. What is bean writing process? Explain the steps used in writing a bean with example.
7. What is meant by bean property? What are its different types? Explain each property briefly.

❑❑❑